

Installation Manual

How to get from a bare ESP32-S3 and a clean machine to a transmitter playing internet radio. Follow it in order. Each step ends with something you can check.

For day-to-day use once it runs, see the User Manual. For how it works inside, see the Technical Manual.

Contents

1. What you need
2. The output network, which is not optional
3. Installing the toolchain
4. Building
5. Flashing
6. First boot and joining a network
7. Checking it works
8. When it goes wrong

1. What you need

Hardware

Item	Notes
ESP32-S3 board	8 MB flash. Verified on an ESP32-S3R8, chip revision v0.2, with 8 MB embedded PSRAM
USB cable	Data, not charge-only
Passives for two output networks	One per carrier. See Â&2 and Design 03 Â&4
An AM receiver	Longwave capable if you keep the default 200 kHz carrier

The board needs no audio hardware, no display and no buttons. Everything is driven from a web page it serves itself.

Your board may expose two USB ports. One is a USB-to-serial bridge (CH343, CP210x or similar). The other is the ESP32-S3's own USB peripheral. **Use the native USB port.** The firmware routes its console there (`CONFIG_ESP_CONSOLE_USB_SERIAL_JTAG`), so the bridge shows you the ROM bootloader banner and nothing else. On Windows the native port appears in

Device Manager as `USB JTAG/serial debug unit` with a `USB Serial Device` COM number.

Host machine

You need Git, Python 3 and about 4 GB of disk. The ESP-IDF installer fetches its own compilers, so you do not need a toolchain first.

Windows users: keep the paths short. `C:\esp` works. A deep path under a user profile can push the build past the Windows path length limit in ways that report themselves as unrelated compiler errors.

2. The output network, which is not optional

Two things stand between this project and a nuisance.

Fit a low-pass network to each carrier pin. The carriers are square waves. A square wave at 200 kHz radiates odd harmonics at 600 kHz, 1.0 MHz, 1.4 MHz and upward, at a third, a fifth and a seventh of the fundamental. Those land across the whole medium-wave broadcast band. A bare wire on GPIO 4 splatters over every station a domestic radio can tune. Design 03 Å\$4 gives a network that works. This is recorded as `KI-02`.

The firmware runs **two** transmitters, on GPIO 4 and GPIO 5 by default. Each needs its own network. Coupling both into one lets them intermodulate, and 200 kHz has its third harmonic at 600 kHz, close enough to a second carrier in the 250 to 300 kHz range that the filtering has to keep them apart rather than merely tidy each one up. That is `KI-08`.

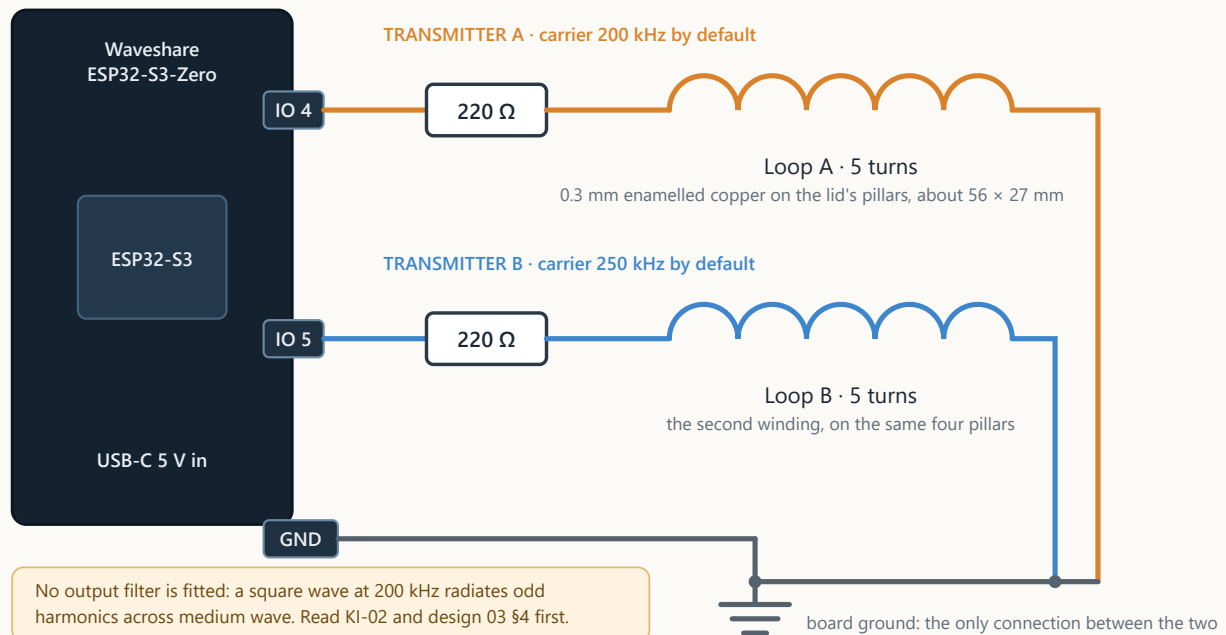
Keep the field in the room. 200 kHz sits inside the longwave broadcast band, 2 kHz from BBC Radio 4 on 198 kHz. Use an inductive loop under or beside the receiver, not an antenna. Deliberate radiation in that band is licensed in the UK and in most other places, and this project holds no licence. What you actually radiate, and the rules where you live, are yours to check. This is recorded as `KI-04`.

Neither point is a formality. Build the network before you connect anything to the pin.

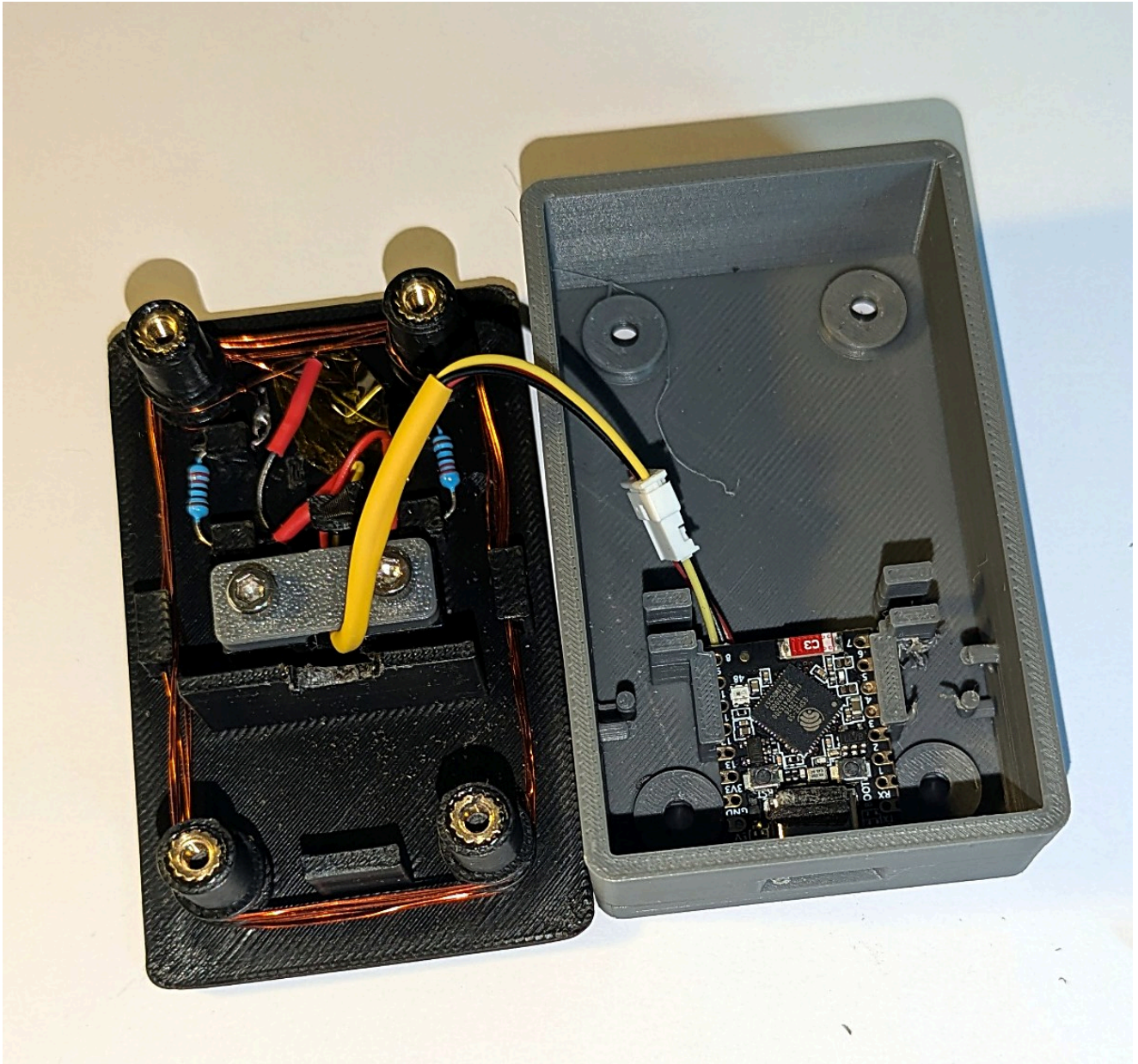
What the built unit actually has

AM transmitter: what is wired to what

Two independent transmitters, one loop each. They share the board's ground and nothing else.



One transmitter per pin: GPIO through a 220 ohm series resistor, into a five-turn loop, back to the board's ground. **No filter**, which is the shortcut described in Design 03 §4a and §4b and not a recommendation. It works at this drive with the receiver in the same room, and nobody has put a spectrum analyser on it. If you build one with more drive, a longer antenna or a smaller resistor, fit the network in Design 03 §4.



The lid carries both windings and both resistors, and the base carries the board. The enclosure is [designs/2026-09-tx-case/](#).

3. Installing the toolchain

The project is pinned to **ESP-IDF v5.3.2**. Other v5.x releases will probably work and have not been tried.

Clone the SDK. A shallow clone is enough and saves a long wait:

```
git clone --depth 1 --recursive --shallow-submodules \  
-b v5.3.2 https://github.com/espressif/esp-idf.git C:/esp/esp-idf
```

Install the tools for this chip:

```
# Windows PowerShell
$env:IDF_PATH = 'C:\esp\esp-idf'
$env:IDF_TOOLS_PATH = 'C:\esp\tools'
C:\esp\esp-idf\install.ps1 esp32s3
```

```
# macOS and Linux
export IDF_PATH=~/.esp/esp-idf
cd $IDF_PATH && ./install.sh esp32s3
```

This downloads roughly 2 GB and takes 15 to 30 minutes. It fetches the Xtensa and RISC-V compilers, CMake, Ninja, OpenOCD, and builds a Python environment.

Check: open a new shell, activate the environment, and ask for a version.

```
# Windows
. C:\esp\esp-idf\export.ps1
# macOS and Linux
. $IDF_PATH/export.sh

idf.py --version      # should print ESP-IDF v5.3.2
```

The environment is per-shell. Run `export` again in every new terminal. This is the single most common cause of `idf.py: command not found`.

4. Building

From the repository root:

```
idf.py set-target esp32s3
idf.py build
```

The first build resolves two managed components from the Espressif registry, `chmorgan/esp-libhelix-mp3` and `espressif/mdns`, then compiles about a thousand files. Allow several minutes. Later builds take seconds.

Check: the build ends with a size line and no warnings.

```
amtx.bin binary size 0x11c0b0 bytes. Smallest app partition is 0x300000 bytes.
0x1e3f50 bytes (63%) free.
Project build complete.
```

You do not need `idf.py menuconfig`. Everything it sets has a working default, and Wi-Fi is configured from the board itself. Use it only to change the RF pin, the audio low-pass, or the two stations a fresh board starts on (Radio Swiss Jazz on A, BBC World Service on B). Anything you enter there lands in `sdkconfig`, which is deliberately not tracked by Git because it can hold a Wi-Fi password.

Host-side tests, no hardware needed

The audio maths builds and runs on your machine:

```
make -C test/host test
```

Twenty-seven checks covering the filters, the resampler, the limiter and the arcsine duty table. They should all pass. On Windows with MinGW, use `mingw32-make CC=gcc`.

5. Flashing

Find the port. On Windows it is the `USB Serial Device` COM number that appears when you connect the native USB port. On Linux it is usually `/dev/ttyACM0`, on macOS `/dev/cu.usbmodem*`.

```
idf.py -p COM18 flash
```

Each of the three images ends with `Hash of data verified`.

Or use the flasher. `python tools/esp-flasher/esp_flasher.py` finds the board, reads what is on it, writes the bootloader, partition table and app from `build/` (or only the app, when the rest already matches), and opens a serial monitor on the board afterwards. It needs no ESP-IDF shell, only `pip install -r tools/esp-flasher/requirements.txt`. See its `README.md`.

A new board that keeps disappearing. A blank ESP32-S3 resets every couple of seconds and drops off USB each time (`LL-06`). The flasher catches it anyway. With `idf.py`, hold BOOT, press and release RESET, release BOOT, then flash: the board stays put in download mode.

If the port is busy: close anything else holding it. A serial terminal keeps the port open, and Windows will not share it. The error is `Could not open COM18, the port is busy or doesn't exist`. Note that a terminal whose window you closed can leave its process running and still holding the port.

6. First boot and joining a network

With no saved network, the board raises its own open access point.

1. Look for a Wi-Fi network called **AMTX-Setup** and join it. It takes about 30 seconds to appear after a reset.
2. A setup page should open by itself. If it does not, browse to `http://192.168.4.1`.
3. Open the **Network** tab and press **Scan for networks**.
4. Pick your network from the list. Using the list rather than typing the name matters: the scan only returns 2.4 GHz networks, and the ESP32-S3 has no 5 GHz radio, so a hand-typed 5 GHz name will never join.
5. Type the password. Press the eye button beside the field to check it before you commit.
6. Press **Save and join**.

A dialog tells you the board is restarting to join. **The setup portal will disconnect at this point, and that is the success path**, not a failure: the access point has to come down before the board can join your network.

Or do it over the USB cable instead. `python tools/amt-x-programmer/amt-x_programmer.py` finds the board, opens its console and gives you the same settings as buttons and dialogs: networks, stations, both transmitters, the hostname and the web interface. It is the only route in once the web interface has been turned off, and it needs no phone.

Reconnect your phone or laptop to your own network and open `http://amt-x.local`. If another transmitter on the network already answers to that name, set this one's hostname on the Network tab, or with `hostname <name>` on the serial console, and use `http://<name>.local` from then on.

If the password was wrong, the board gives up after about 15 seconds per saved network and brings `AMTX-Setup` back, so you can rejoin and try again.

If mDNS does not work

Some networks and some operating systems do not resolve `.local` names. Find the address instead. Connect the native USB port, open a serial terminal at 115200 baud, and wait up to 30 seconds:

```
I (30300) wifi: wifi up: connected to "YourNetwork", ip 192.168.0.123, rssi -57 dBm, drops 0
```

That heartbeat prints every 30 seconds whether or not anything has changed, so silence means nothing is running rather than nothing has changed.

You can also ask the board directly. Press Enter in the terminal to get the `amtx>` prompt and type `status`.

7. Checking it works

Work down this list. Each line is something you can see.

Check	What you should see
It boots	<code>webui: web interface up on port 80</code>
It joined	<code>wifi up: connected to "...", ip ... every 30 s</code>
The web UI answers	<code>http://amtx.local</code> loads the page
The carrier runs	Header reads <code>ON AIR 200.0 kHz 70%</code>
A stream plays	Now playing shows a station and <code>playing</code>
Audio decodes	Diagnostics shows decoded frames climbing, roughly 40 per second
The modulator is fed	Modulator FIFO near half full, underruns not climbing
A radio hears it	Tune the receiver to the carrier frequency

The last line is the only one this project has never checked. Nothing here has been measured off-air: no receiver, no spectrum analyser, no field-strength reading. The signal chain is verified up to the duty register and no further.

If it is set up wrong and you cannot reach it

Hold the board's **BOOT** button (marked **B**) for five seconds while it is running. It forgets every station, every saved network, both transmitters' settings, its hostname and whether the web interface runs, then restarts and raises `AMTX-Setup` again. The firmware stays; this only clears what the board remembers. User Manual §6 has the detail.

That is the answer to the two states nothing else recovers: a board whose only saved network no longer exists, and a board whose web interface was turned off on a network you can no longer join.

8. When it goes wrong

`idf.py` **not found**. The environment is per-shell. Run the export script again.

Build fails on a missing component. The first build needs to reach `components.espressif.com`. Behind a restrictive proxy it cannot, and the error

does not say so. LL-03 records the workaround.

Nothing on the serial port but a ROM banner. You are on the USB-to-serial bridge, not the native USB port. Move the cable.

The port is busy. Something else has it open. See Â§5.

The port keeps appearing and vanishing. The flash is blank or corrupt and the chip is boot-looping. Flash it; see Â§5 and LL-06 .

The setup portal will not take a password. Firmware before the KI-07 fix reset itself on every settings write, so no credential could ever be saved. Check you are running a current build. The symptom is a board that joins your network correctly and then reappears on AMTX-Setup after a reboot with nothing saved.

A station will not play. Check Diagnostics for the HTTP status. 302 on a current build means the redirect chain ran past its limit. Anything that is not MP3 will connect and then produce silence, because libhelix is the only decoder compiled in.

Your browser keeps probing. While you sit on AMTX-Setup , phones and browsers poll for a captive portal every few seconds. That is normal and stops once the board has real internet.